

Analítica de Negocios

Parcial Final 20%

ANÁLISIS BASE DE DATOS “WINE QUALITY”

David Ocampo Sáenz





CONTEXTO DE LA BASE DE DATOS

“WINE QUALITY”

La base de datos recopila información sobre las características físico-químicas de vinos tintos y blancos, junto con una puntuación de calidad asignada por catadores expertos.

Su propósito es permitir el análisis de cómo diversos factores como el contenido de alcohol, la acidez, los sulfatos y el azúcar residual se relacionan con la calidad final del vino.

Tipos de VARIABLES

Categóricas explicativas

- type: Indica el tipo de vino que se está analizando. Valores posibles: Red (Tinto) / White (Blanco).

Numéricas explicativas

- fixed acidity
- volatile acidity
- citric acid
- residual sugar
- chlorides
- free sulfur dioxide
- total sulfur dioxide
- density
- pH
- sulphates
- alcohol

Explicativa numérica de pronóstico

- quality: Puntuación de calidad asignada por catadores expertos. En el modelo se convierte en binaria: 0 = Mala calidad (≤ 5) / 1 = Buena calidad (≥ 6).

Representan las propiedades físico-químicas del vino, agrupadas en cuatro categorías: acidez, composición química, conservante y características físicas.

Modelos usados para el **ANÁLISIS**

**Naive
Bayes**

**Clustering
K-Means**



¿POR QUÉ ES IMPORTANTE PARA UNA EMPRESA VITIVINÍCOLA?

Una empresa vitivinícola puede usar estos modelos para:

- Predecir automáticamente si un vino será de buena o mala calidad antes de lanzarlo al mercado.
- Identificar qué características químicas definen un vino de alta calidad.
- Segmentar su portafolio de vinos para estrategias de precios y mercadeo.
- Optimizar el proceso de producción reduciendo costos de catadores humanos.



NAIVE BAYES

¿Por qué Naive Bayes?

- El problema es de clasificación binaria: queremos predecir entre dos categorías de calidad del vino (Buena calidad y Mala calidad).
- Las variables predictoras son tanto numéricas como categóricas (tipo de vino), y Naive Bayes puede adaptarse bien a este tipo de datos.
- Es un modelo rápido de entrenar y fácil de interpretar.

¿Qué predice y con qué datos?

- **Variable objetivo:** Calidad del vino (Buena o Mala)
- **Variables usadas:**
 - Type (tipo de vino)
 - Alcohol
 - Volatile Acidity (acidez volátil)
 - Sulphates (sulfatos)
 - Citric Acid (ácido cítrico)
 - Residual Sugar (azúcar residual)
 - pH (grado de acidez)
- **Objetivo general:** Clasificar correctamente la calidad de un vino entre buena o mala, en función de sus características fisicoquímicas y su tipo, con el fin de apoyar las decisiones de producción y control de calidad de una empresa vitivinícola.

IMPLEMENTACIÓN DEL MODELO

Librerías y base de datos

```
1 # Librerías necesarias
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 from sklearn.preprocessing import LabelEncoder, StandardScaler
8 from sklearn.model_selection import train_test_split
9 from sklearn.naive_bayes import GaussianNB
10 from sklearn.cluster import KMeans
11 from sklearn.metrics import (
12     confusion_matrix, classification_report,
13     accuracy_score, precision_score, recall_score
14 )
```

```
1 # Subimos la base de datos
2 df = pd.read_csv('/content/winequalityN.csv')
```


IMPLEMENTACIÓN DEL MODELO

Adaptaciones para usar
base de datos

```
1 # Crear variable binaria de calidad: 0 = Malo (quality <= 5), 1 = Bueno (quality >= 6)
2 df['quality_label'] = (df['quality'] >= 6).astype(int)
3
4 print('Distribución de calidad:')
5 print(df['quality_label'].value_counts())
6 print('\n0 = Vino de baja calidad (quality <= 5)')
7 print('1 = Vino de buena calidad (quality >= 6)')
```

Distribución de calidad:

quality_label

1 4113

0 2384

Name: count, dtype: int64

0 = Vino de baja calidad (quality <= 5)

1 = Vino de buena calidad (quality >= 6)

```
1 # Codificar variable categórica 'type' (red=1, white=0)
2 le_type = LabelEncoder()
3 df['type_encoded'] = le_type.fit_transform(df['type'])
4
5 print('Mapeo de tipo de vino:')
6 for i, clase in enumerate(le_type.classes_):
7     print(f'{i} → {clase}')
```

Mapeo de tipo de vino:

0 → red

1 → white

IMPLEMENTACIÓN DEL MODELO

Preparación de los datos

```
1 columnas_entrada = ['type_encoded', 'alcohol', 'volatile acidity',
2 | | | | | | | | | | 'sulphates', 'citric acid', 'residual sugar', 'pH']
3
4 XDB = df[columnas_entrada + ['quality_label']].dropna()
5
6 # Variables de entrada y salida
7 XD = XDB[columnas_entrada] # Variables de entrada
8 YD = XDB[['quality_label']] # Variable de salida (pronóstico)
9
10 print('Variables de entrada (X):', list(XD.columns))
11 print('Variable de salida (Y):', list(YD.columns))
```

```
Variables de entrada (X): ['type_encoded', 'alcohol', 'volatile acidity', 'sulphates', 'citric acid', 'residual sugar', 'pH']
Variable de salida (Y): ['quality_label']
```

```
1 # 2. Codificar la variable de salida
2 le = LabelEncoder()
3 YD_encoded = le.fit_transform(YD.values.ravel())
4
5 # Mostrar mapeo entre clases y números
6 print('Mapeo de categorías de quality_label:')
7 for i, clase in enumerate(le.classes_):
8 | | print(f'{i} → {"Baja calidad" if clase == 0 else "Buena calidad"}')
```

```
Mapeo de categorías de quality_label:
0 → Baja calidad
1 → Buena calidad
```


IMPLEMENTACIÓN DEL MODELO

Entrenamiento del modelo

```
2 # 4. Entrenamiento del modelo Naive Bayes
3
4 mnb = GaussianNB()
5 mnb.fit(XD_train, YD_train)
6
7 # Información del modelo
8 u = mnb.theta_
9 print('La media de las variables de entrada es:\n', u)
10
11 var = mnb.var_
12 print('\nLa varianza de las variables de entrada es:\n', var)
13
14 sigma = np.sqrt(var)
15 print('\nLa desviación estándar es:\n', sigma)
16
17 LS = u + sigma
18 LI = u - sigma
19 print('\nLímites superiores:\n', LS)
20
21 print('Límites inferiores:\n', LI)
```


IMPLEMENTACIÓN DEL MODELO

Ejemplo de predicción

```
1 # 5. Ejemplo de predicción para un vino nuevo
2 # [type_encoded, alcohol, volatile acidity, sulphates, citric acid, residual sugar, pH]
3 # type_encoded: 0 = white, 1 = red
4 XDS = [[1, 12.5, 0.3, 0.7, 0.4, 2.5, 3.2]] # Vino tinto de ejemplo
5 YDP = mnb.predict(XDS)
6
7 # Obtener el nombre del diagnóstico
8 nombre_calidad = le.inverse_transform(YDP)
9
10 # Mostrar resultados
11 print('La predicción del vino es:')
12 print(f'Código: {YDP[0]}')
13 print(f'Calidad: {"Buena calidad" if nombre_calidad[0] == 1 else "Baja calidad"}')
```

La predicción del vino es:

Código: 1

Calidad: Buena calidad

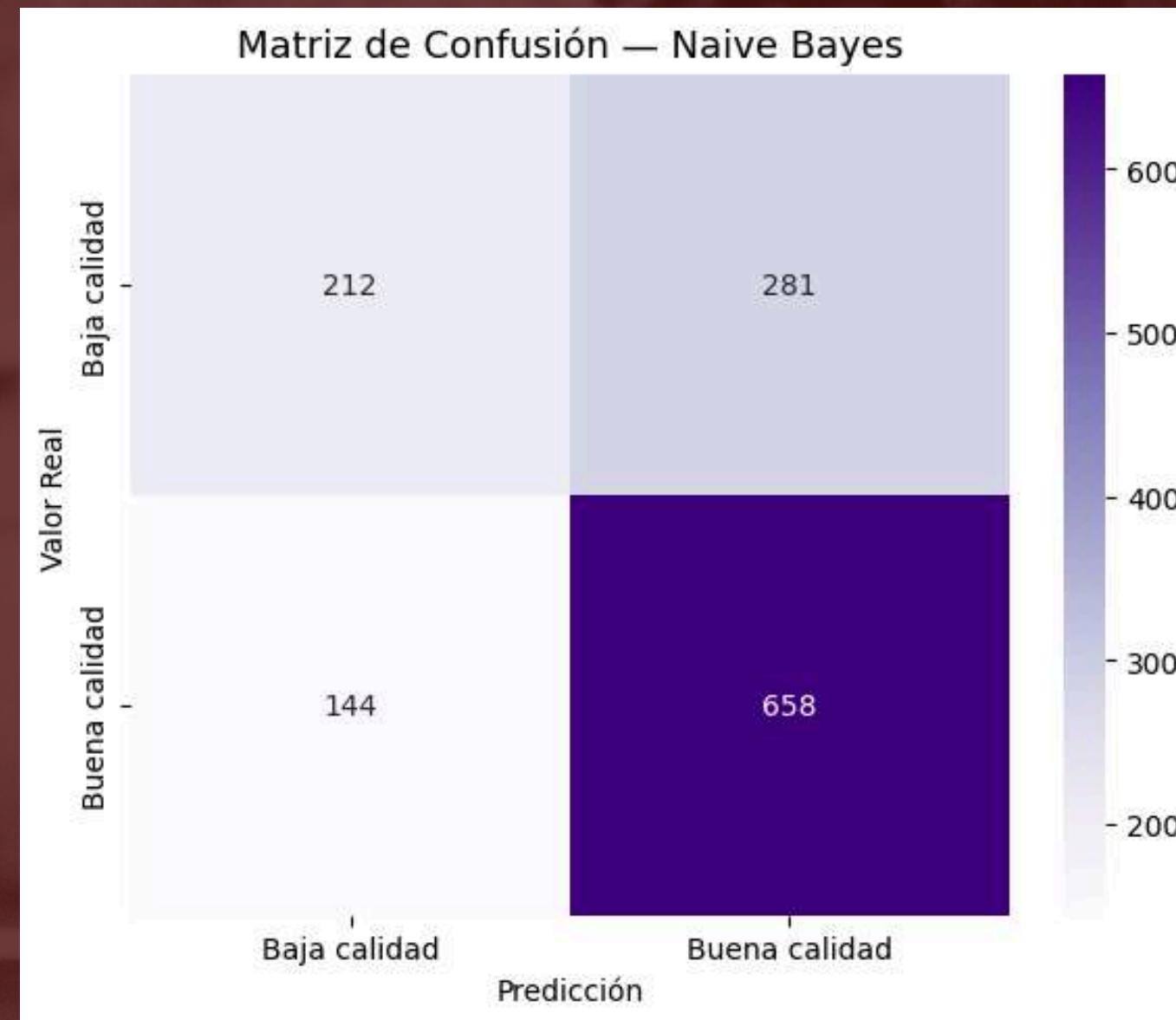
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does
warnings.warn(

IMPLEMENTACIÓN DEL MODELO

Evaluación del modelo

```
1 # 6. Evaluación del modelo
2
3 # Identificar filas con NaN en XD_test
4 nan_rows_mask_test = XD_test.isnull().any(axis=1)
5
6 # Crear versiones limpias de XD_test y YD_test eliminando las filas con NaN
7 XD_test_cleaned = XD_test[~nan_rows_mask_test]
8 YD_test_cleaned = YD_test[~nan_rows_mask_test]
9
10 YDP2 = mnbc.predict(XD_test_cleaned)
11
12 # Matriz de confusión
13 cm = confusion_matrix(YD_test_cleaned, YDP2)
14 print('La matriz de confusión es:\n', cm)
15
16 # Reporte de clasificación completo
17 print('\nReporte de clasificación (por clase):')
18 print(classification_report(YD_test_cleaned, YDP2,
19 | | | target_names=['Baja calidad', 'Buena calidad']))
20
21 # Métricas generales
22 exactitud = accuracy_score(YD_test_cleaned, YDP2)
23 print('Exactitud del modelo:', round(exactitud * 100, 1), '%')
24
25 precision_prom = precision_score(YD_test_cleaned, YDP2, average='macro')
26 print('Precisión promedio (macro):', round(precision_prom * 100, 1), '%')
27
28 sensibilidad_prom = recall_score(YD_test_cleaned, YDP2, average='macro')
29 print('Sensibilidad promedio (macro):', round(sensibilidad_prom * 100, 1), '%')
```


MATRIZ DE CONFUSIÓN



El modelo tiene un comportamiento asimétrico, es bastante bueno identificando vinos de buena calidad (acierta el 82% de ellos), pero es débil con los vinos de baja calidad (solo acierta el 43%). Esto se debe a que en el Wine Quality Dataset hay más vinos de buena calidad que de mala, y Naive Bayes tiende a favorecer la clase que aparece más en el entrenamiento.

REPORTE DE CLASIFICACIÓN

Reporte de clasificación (por clase):

	precision	recall	f1-score	support
Baja calidad	0.60	0.43	0.50	493
Buena calidad	0.70	0.82	0.76	802

Métricas globales:

Exactitud total (accuracy): 67.2%

Precisión promedio (macro): 64.8%

Recall promedio (macro): 62.5%

CONCLUSIONES Y ANÁLISIS

Fortalezas del modelo

- Exactitud global del 67.2%, razonable para Naive Bayes con datos continuos.
- Recall del 82% en "Buena calidad": detecta bien los vinos buenos.
- Cuando predice "Buena calidad", acierta el 70% de las veces.

Mejoras del modelo

- Solo acierta el 43% de los vinos de baja calidad
- El desbalance de clases (493 malos vs. 802 buenos) sesga el modelo hacia la clase mayoritaria.
- La correlación entre variables como alcohol y densidad viola el supuesto de independencia de Naive Bayes.

Conclusión

Naive Bayes es una base sólida para clasificar calidad de vino, pero su debilidad con la clase minoritaria limita su aplicación real. Modelos como Random Forest o SVM podrían mejorar significativamente los resultados.

K-MEANS

¿Por qué K-Means?

El modelo de K-Means es un método de aprendizaje no supervisado utilizado para descubrir grupos o patrones ocultos en los datos, sin conocer previamente las clases o categorías.

En el caso de los vinos, es ideal porque permite segmentar automáticamente grandes volúmenes de vinos en grupos con perfiles físico-químicos similares, sin necesidad de que un experto los clasifique manualmente, lo que facilita la toma de decisiones.

Objetivo del análisis

Detectar grupos de vinos con perfiles físico-químicos similares, con el fin de segmentar el portafolio de productos de una empresa vitivinícola y apoyar decisiones de producción, precios y marketing.

- **Variables usadas (solo numéricas):**

- Alcohol
- Volatile Acidity (acidez volátil)
- Sulphates (sulfatos)
- Citric Acid (ácido cítrico)
- Residual Sugar (azúcar residual)
- pH (grado de acidez)
- Density (densidad)

IMPLEMENTACIÓN DEL MODELO

Preparación de los datos

```
1 # 1. Selección de variables para K-Means (solo numéricas)
2 variables_kmeans = ['alcohol', 'volatile acidity', 'sulphates',
3 | | | | | | | | | | 'citric acid', 'residual sugar', 'pH', 'density']
4
5 XDB_km = df[variables_kmeans + ['quality_label']].dropna()
6
7 # Separar variables predictoras y objetivo
8 XD_km = XDB_km[variables_kmeans]
9 YD_km = XDB_km[['quality_label']]
10
11 # Codificar variable de salida
12 le_km = LabelEncoder()
13 YD_km_encoded = le_km.fit_transform(YD_km.values.ravel())
14
15 print('Mapeo de categorías de quality_label:')
16 for i, clase in enumerate(le_km.classes_):
17 | | print(f'{i} → {"Baja calidad" if clase == 0 else "Buena calidad"}')
```

Mapeo de categorías de quality_label:

0 → Baja calidad

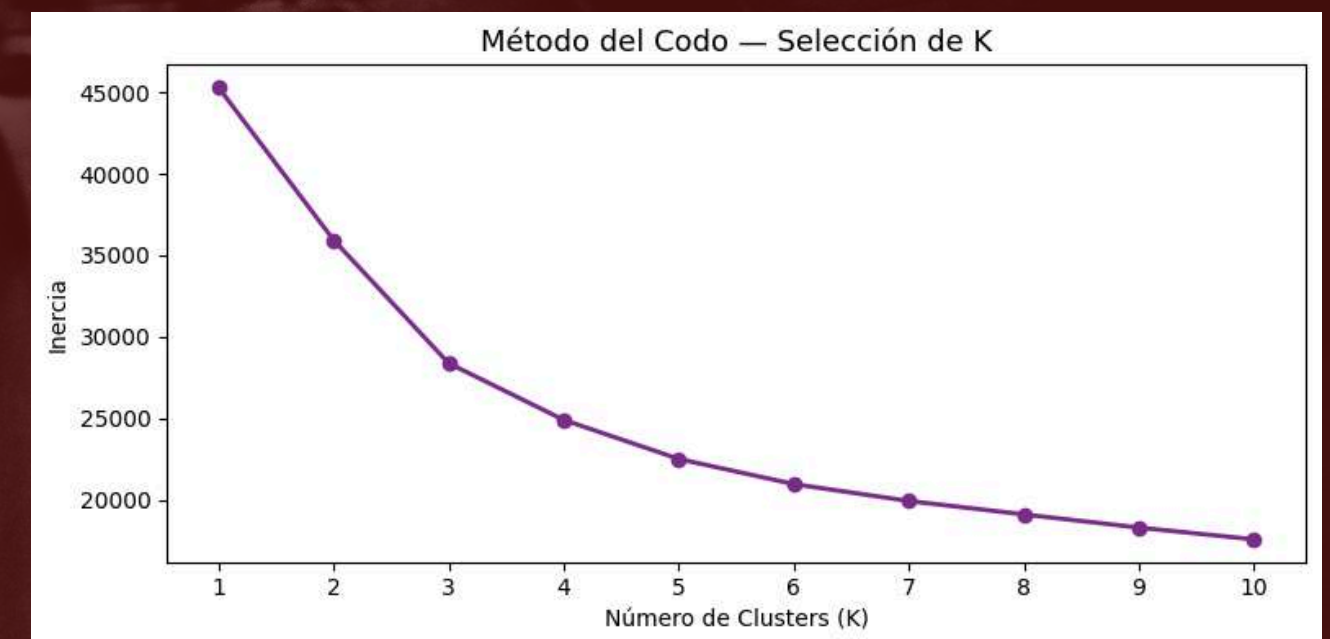
1 → Buena calidad

IMPLEMENTACIÓN DEL MODELO

Método del codo para elegir clusters

```
1 # 3. Método del Codo para elegir el número óptimo de clusters
2 inercia = []
3 rango_k = range(1, 11)
4
5 for k in rango_k:
6     km_temp = KMeans(n_clusters=k, random_state=42, n_init=10)
7     km_temp.fit(XD_km_scaled)
8     inercia.append(km_temp.inertia_)
9
10 plt.figure(figsize=(8, 4))
11 plt.plot(rango_k, inercia, marker='o', color='#7B2D8B', linewidth=2)
12 plt.title('Método del Codo - Selección de K', fontsize=13)
13 plt.xlabel('Número de Clusters (K)')
14 plt.ylabel('Inercia')
15 plt.xticks(rango_k)
16 plt.tight_layout()
17 plt.show()
```

****Prueba el modelo con diferentes valores de K (1, 2, 3, 4... grupos) y mide qué tan compactos quedan los grupos. Cuando se grafica esto, la curva forma una especie de codo y el punto donde se dobla te indica el K óptimo****



IMPLEMENTACIÓN DEL MODELO

Entrenamiento del modelo

```
1 # 4. Crear modelo K-Means con K=3
2 np.random.seed(42)
3 nc = 3 # Número de clusters
4 mkm = KMeans(n_clusters=nc, random_state=42, n_init=10)
5 mkm.fit(XD_km_scaled)
6
7 # Centroides
8 centroides = mkm.cluster_centers_
9 print('Centroides de los clusters:\n', centroides)
10
11 # Etiquetas de cluster para cada muestra
12 labels = mkm.labels_
13 XDB_km = XDB_km.copy()
14 XDB_km['Cluster'] = labels
15
16 # Conteo de vinos por cluster
17 ND = np.bincount(labels)
18 print('\nNúmero de vinos por cluster:\n', ND)
```


IMPLEMENTACIÓN DEL MODELO

Entrenamiento del modelo

```
1 # 5. Distribución de calidad por cluster
2 NBaja = XDB_km.groupby('Cluster').agg({'quality_label': lambda x: (x == 0).sum()})
3 NBuena = XDB_km.groupby('Cluster').agg({'quality_label': lambda x: (x == 1).sum()})
4
5 NBaja['Baja calidad %'] = NBaja['quality_label'] / ND * 100
6 NBuena['Buena calidad %'] = NBuena['quality_label'] / ND * 100
7
8 print('\nPorcentaje de vinos de BAJA calidad por cluster:\n', NBaja[['Baja calidad %']].round(1))
9 print('\nPorcentaje de vinos de BUENA calidad por cluster:\n', NBuena[['Buena calidad %']].round(1))
```

```
1 # 6. Análisis de centroides en escala original
2 centroides_original = scaler.inverse_transform(centroides)
3 df_centroides = pd.DataFrame(centroides_original, columns=variables_kmeans)
4 df_centroides.index.name = 'Cluster'
5 print('\nCentroides en escala original:')
6 print(df_centroides.round(3))
```


IMPLEMENTACIÓN DEL MODELO

Ejemplo de predicción

```
1 # 9. Ejemplo de predicción: asignar cluster a un vino nuevo
2 # Ajustar cluster_to_class según el análisis de los centroides
3 cluster_to_class = {0: 1, 1: 0, 2: 1} # Ajustar según resultados del codo
4
5 ydp = np.zeros(len(XD_km), dtype=int)
6 for k in range(len(XD_km)):
7     cluster = labels[k]
8     ydp[k] = cluster_to_class[cluster]
9
10 # Evaluar desempeño
11 cm_km = confusion_matrix(YD_km_encoded, ydp)
12 print('Matriz de confusión K-Means:\n', cm_km)
13
14 print('\nReporte de clasificación:')
15 print(classification_report(YD_km_encoded, ydp,
16     target_names=['Baja calidad', 'Buena calidad']))
17
18 print('Exactitud global:', round(accuracy_score(YD_km_encoded, ydp) * 100, 1), '%')
```

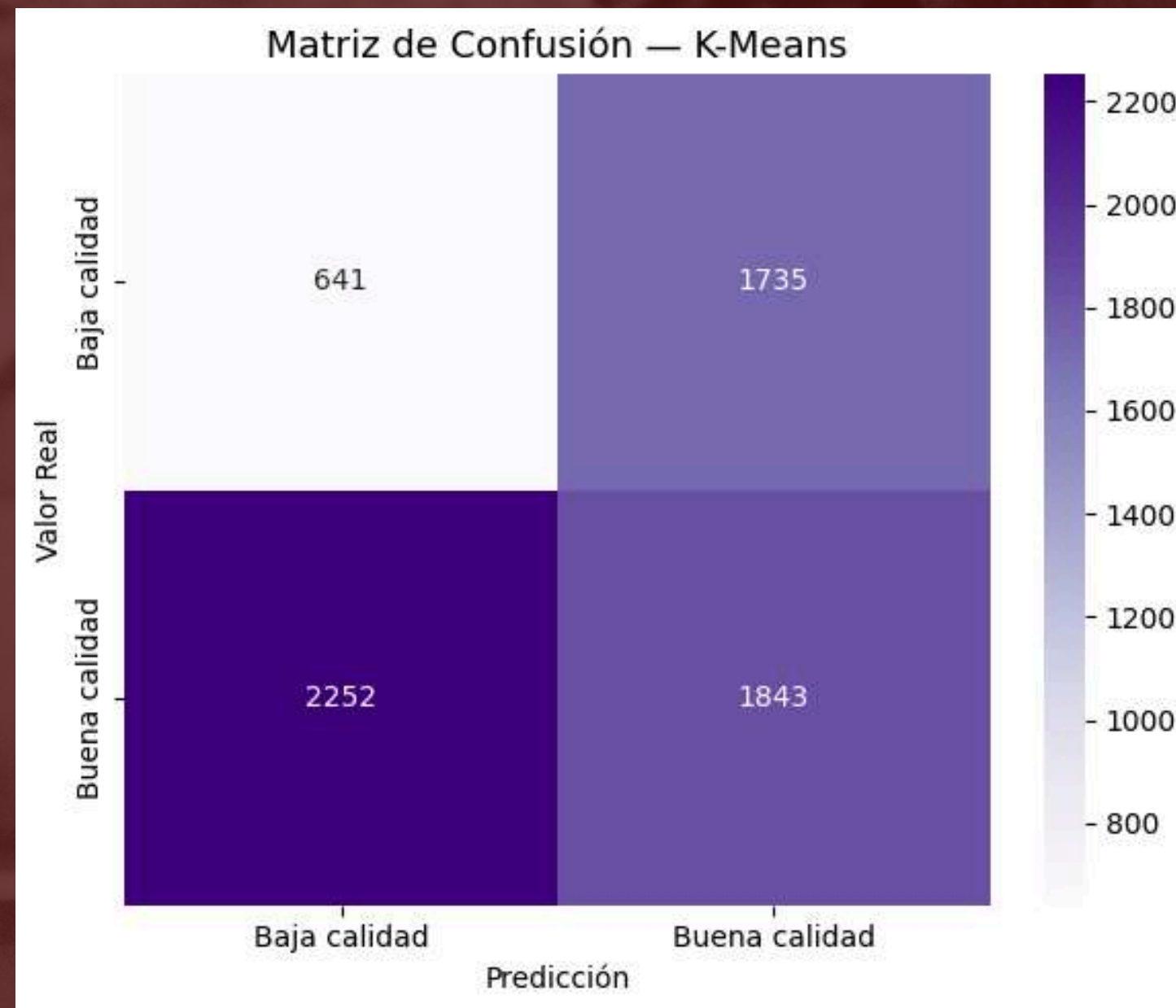

ANÁLISIS DETALLADO DE LOS RESULTADOS

CENTROIDES DE LOS CLUSTERS

- Cluster 0: acidez alta y alcohol medio → vinos secos de calidad intermedia, posiblemente tintos.
- Cluster 1: alcohol alto y baja acidez → vinos de mejor calidad, perfil más limpio y equilibrado.
- Cluster 2: azúcar muy alto y alcohol bajo → vinos dulces o semidulces, probablemente blancos.

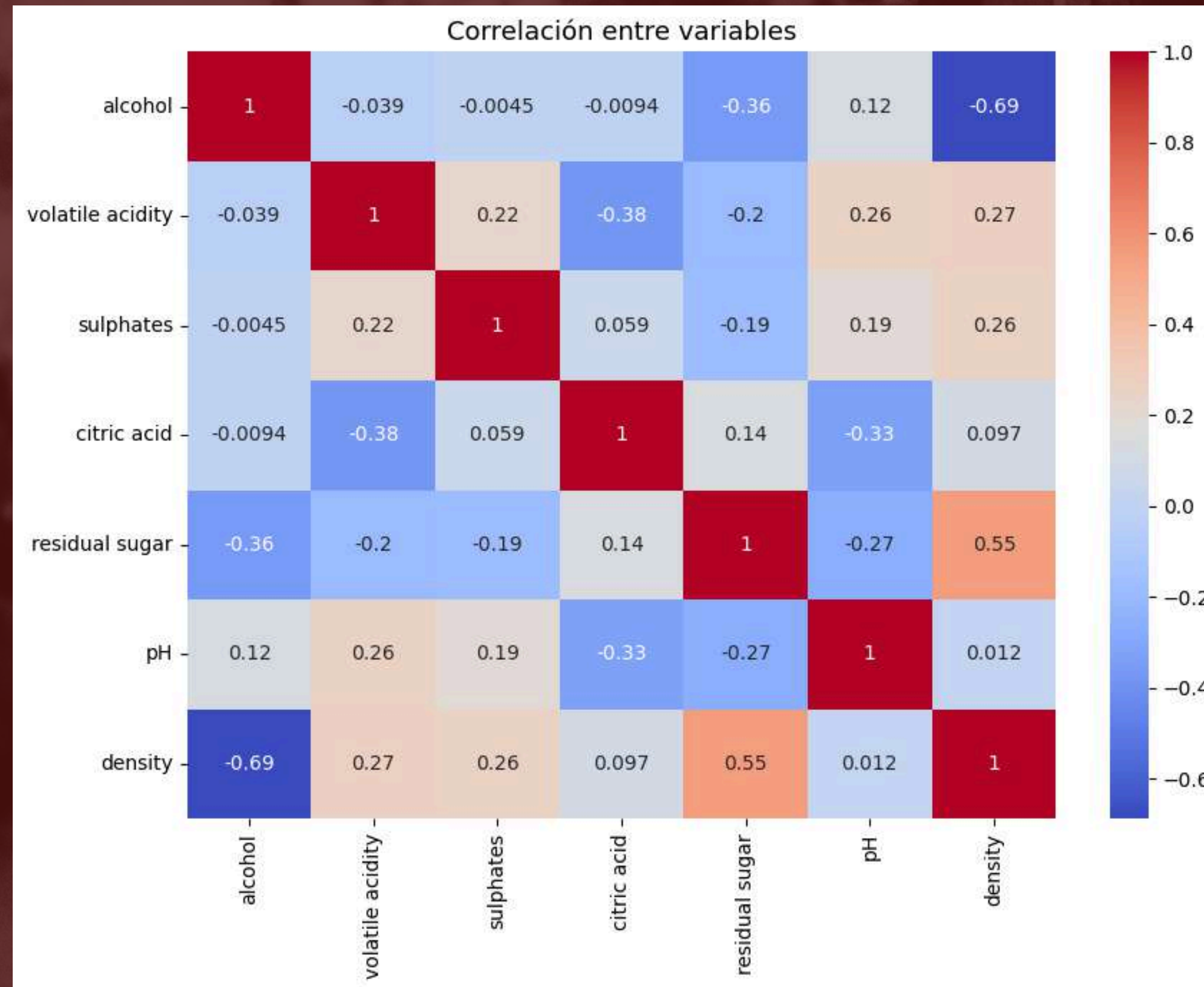
Cluster	alcohol	volatile acidity	sulphates	citric acid	residual sugar	pH	density
0	10.227	0.544	0.649	0.223	2.722	3.336	0.996
1	11.277	0.269	0.494	0.337	3.126	3.203	0.992
2	9.578	0.283	0.494	0.366	10.858	3.149	0.997

MATRIZ DE CONFUSIÓN



K-Means tiene un desempeño muy pobre como clasificador, apenas acierta el 38.4% de los casos. Esto era esperable, pues K-Means es un modelo de agrupamiento no supervisado y no fue diseñado para clasificar. Los clusters se forman por similitud química, no por la etiqueta de calidad.

MATRIZ DE CORRELACIÓN



MÉTRICAS CLAVE DEL MODELO

- Baja calidad es muy difícil de identificar para el modelo: precisión del 22% y recall del 27%, el peor desempeño de ambas clases. K-Means apenas logra agrupar correctamente los vinos malos.
- Buena calidad tiene un desempeño algo mejor pero igualmente bajo: precisión del 52% y recall del 45%. El modelo acierta la mitad de las veces cuando predice un vino como bueno.
- Ninguna clase tiene un equilibrio aceptable, confirmando que K-Means no es adecuado para clasificación supervisada en este dataset.

Reporte de clasificación:

	precision	recall	f1-score
Baja calidad	0.22	0.27	0.24
Buena calidad	0.52	0.45	0.48

CONCLUSIONES Y ANÁLISIS

Fortalezas del modelo

- Los clusters identifican perfiles reales y diferenciables: vinos secos tintos, vinos equilibrados de buena calidad y vinos blancos dulces.
- Útil para segmentar el portafolio de productos sin necesidad de catadores expertos.

Mejoras del modelo

- Exactitud del 38.4% como clasificador, muy por debajo de Naive Bayes (67.2%), confirmando que no fue diseñado para clasificar.
- Los clusters se forman por similitud química, pero no coinciden con la percepción subjetiva de calidad de los catadores.

Conclusión

K-Means es una herramienta valiosa para segmentación y exploración del portafolio de vinos, no para predecir calidad. Su verdadero aporte está en identificar grupos con características físico-químicas similares, lo que puede apoyar decisiones de marketing, precios y producción.

¡MUCHAS GRACIAS!

